

LAB PROGRAM

CHAPTER 3

1. Write a Program to find both the maximum and minimum values in the array.

Implement using any

programming language of your choice. Execute your code and provide the maximum and

minimum values found.

CODE

```
arr = [5, 7, 3, 4, 9, 12, 6, 2]
```

```
maximum = arr[0]
```

```
minimum = arr[0]
```

```
for i in arr:
```

```
    if i > maximum:
```

```
        maximum = i
```

```
    if i < minimum:
```

```
        minimum = i
```

```
print("Min =", minimum)
```

```
print("Max =", maximum)
```

OUTPUT

main.py	   Share 	Output
<pre>1 # Find Maximum and Minimum 2 3 arr = [5, 7, 3, 4, 9, 12, 6, 2] 4 5 maximum = arr[0] 6 minimum = arr[0] 7 8 for i in arr: 9 if i > maximum: 10 maximum = i 11 if i < minimum: 12 minimum = i 13 14 print("Min =", minimum) 15 print("Max =", maximum)</pre>		<pre>Min = 2 Max = 12 === Code Executed ===</pre>

2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to

find both the maximum and minimum values in the array. Implement using any programming

language of your choice. Execute your code and provide the maximum and minimum values found.

CODE

```
arr = [2, 4, 6, 8, 10, 12, 14, 18]
minimum = arr[0]
maximum = arr[-1]
print("Min =", minimum)
print("Max =", maximum)
```

OUTPUT

main.py	Run	Output
<pre>1 # Since array is sorted 2 3 arr = [2, 4, 6, 8, 10, 12, 14, 18] 4 5 minimum = arr[0] 6 maximum = arr[-1] 7 8 print("Min =", minimum) 9 print("Max =", maximum)</pre>		<pre>Min = 2 Max = 18 === Code E</pre>

3. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and

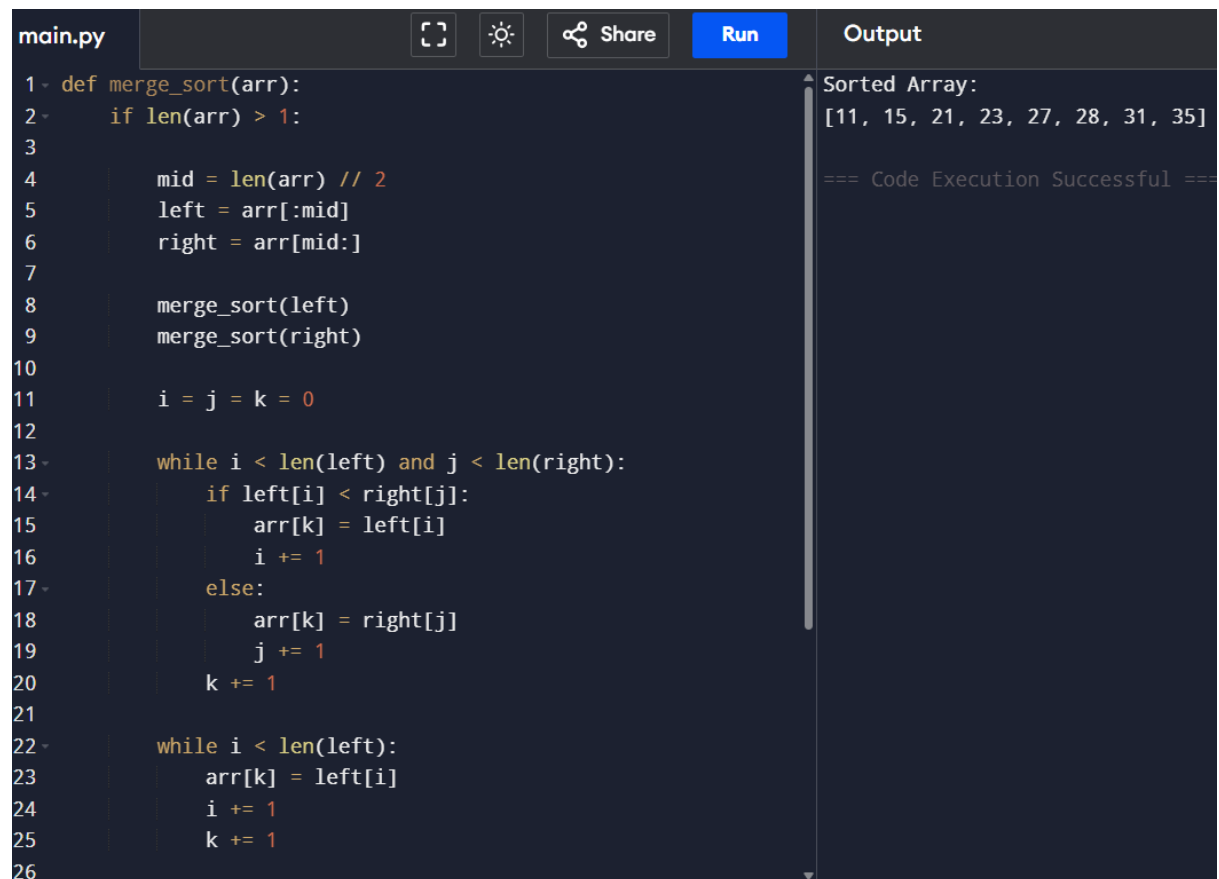
implement using any programming language of your choice.

CODE

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
```

```
merge_sort(left)
merge_sort(right)
i = j = k = 0
while i < len(left) and j < len(right):
    if left[i] < right[j]:
        arr[k] = left[i]
        i += 1
    else:
        arr[k] = right[j]
        j += 1
    k += 1
while i < len(left):
    arr[k] = left[i]
    i += 1
    k += 1
while j < len(right):
    arr[k] = right[j]
    j += 1
    k += 1
arr = [31,23,35,27,11,21,15,28]
merge_sort(arr)
print("Sorted Array:")
print(arr)
```

OUTPUT



The screenshot shows a code editor with a file named 'main.py'. The code implements a merge sort algorithm. The output panel on the right shows the sorted array [11, 15, 21, 23, 27, 28, 31, 35] and a success message.

```
main.py
1 def merge_sort(arr):
2     if len(arr) > 1:
3
4         mid = len(arr) // 2
5         left = arr[:mid]
6         right = arr[mid:]
7
8         merge_sort(left)
9         merge_sort(right)
10
11     i = j = k = 0
12
13     while i < len(left) and j < len(right):
14         if left[i] < right[j]:
15             arr[k] = left[i]
16             i += 1
17         else:
18             arr[k] = right[j]
19             j += 1
20         k += 1
21
22     while i < len(left):
23         arr[k] = left[i]
24         i += 1
25         k += 1
26
```

Output

Sorted Array:
[11, 15, 21, 23, 27, 28, 31, 35]

=== Code Execution Successful ===

4. Implement the Merge Sort algorithm in a programming language of your choice and test it on the

array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons

made during the sorting process. Print this count along with the sorted array.

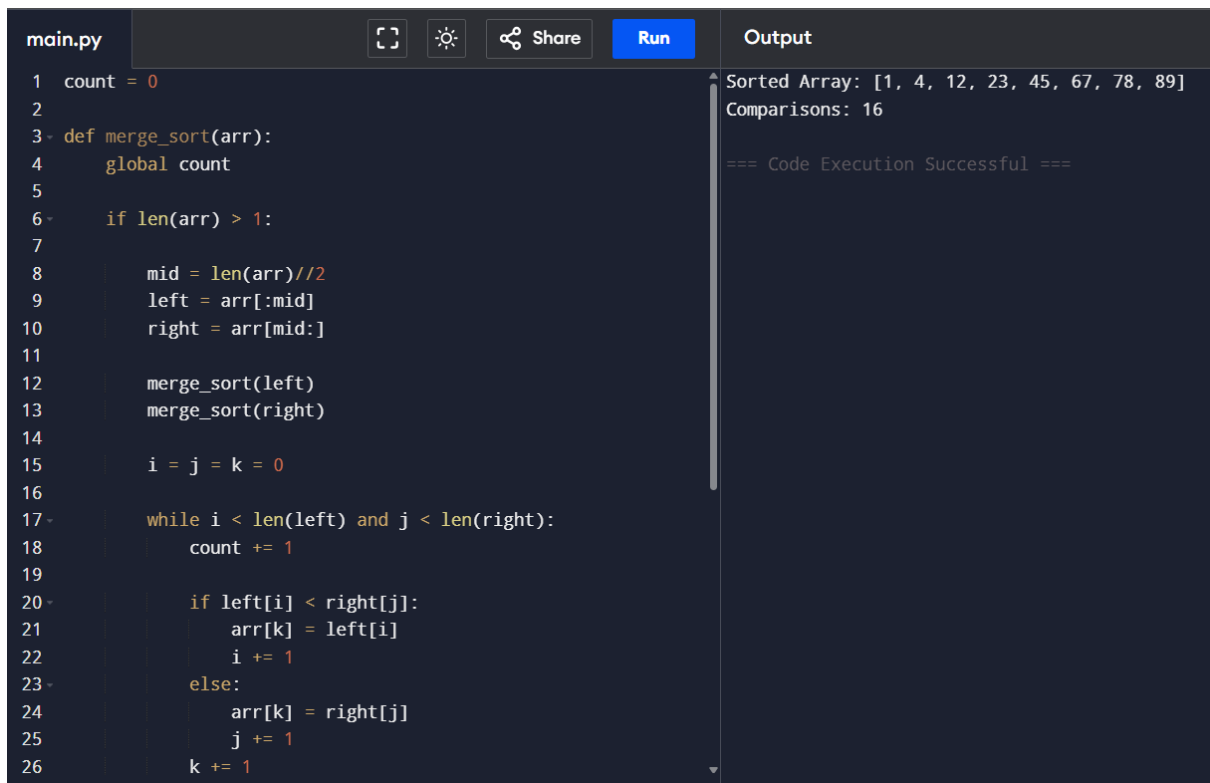
CODE

```
count = 0

def merge_sort(arr):
    global count
    if len(arr) > 1:
        mid = len(arr)//2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
```

```
i = j = k = 0
while i < len(left) and j < len(right):
    count += 1
    if left[i] < right[j]:
        arr[k] = left[i]
        i += 1
    else:
        arr[k] = right[j]
        j += 1
    k += 1
while i < len(left):
    arr[k] = left[i]
    i += 1
    k += 1
while j < len(right):
    arr[k] = right[j]
    j += 1
    k += 1
arr = [12,4,78,23,45,67,89,1]
merge_sort(arr)
print("Sorted Array:", arr)
print("Comparisons:", count)
```

OUTPUT



```
main.py  [Icons]  Run  Output
1 count = 0
2
3 def merge_sort(arr):
4     global count
5
6     if len(arr) > 1:
7
8         mid = len(arr)//2
9         left = arr[:mid]
10        right = arr[mid:]
11
12        merge_sort(left)
13        merge_sort(right)
14
15        i = j = k = 0
16
17        while i < len(left) and j < len(right):
18            count += 1
19
20            if left[i] < right[j]:
21                arr[k] = left[i]
22                i += 1
23            else:
24                arr[k] = right[j]
25                j += 1
26            k += 1
```

Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16
=== Code Execution Successful ===

5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the

first element as the pivot and partition the array accordingly. Show the array after this partition.

Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call

until the entire array is sorted.

CODE

```
def partition(arr, low, high):
    pivot = arr[low]
    i = low + 1
    j = high
    while True:
        while i <= j and arr[i] <= pivot:
            i += 1
        while i <= j and arr[j] > pivot:
            j -= 1
```

```

        if i <= j:
            arr[i], arr[j] = arr[j], arr[i]
        else:
            break
    arr[low], arr[j] = arr[j], arr[low]
    return j

def quick_sort(arr, low, high):
    if low < high:
        p = partition(arr, low, high)
        print("After Partition:", arr)
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)

arr = [10,16,8,12,15,6,3,9,5]
print("Original Array:", arr)
quick_sort(arr, 0, len(arr)-1)
print("Sorted Array:", arr)

```

OUTPUT

main.py	Run	Output
<pre> 1- def partition(arr, low, high): 2- pivot = arr[low] 3- i = low + 1 4- j = high 5- 6- while True: 7- while i <= j and arr[i] <= pivot: 8- i += 1 9- while i <= j and arr[j] > pivot: 10- j -= 1 11- if i <= j: 12- arr[i], arr[j] = arr[j], arr[i] 13- else: 14- break 15- 16- arr[low], arr[j] = arr[j], arr[low] 17- return j 18- 19- def quick_sort(arr, low, high): 20- if low < high: 21- p = partition(arr, low, high) 22- print("After Partition:", arr) 23- quick_sort(arr, low, p - 1) 24- quick_sort(arr, p + 1, high) 25- 26- arr = [10,16,8,12,15,6,3,9,5] </pre>	Run	<pre> Original Array: [10, 16, 8, 12, 15, 6, 3, 9, 5] After Partition: [6, 5, 8, 9, 3, 10, 15, 12, 16] After Partition: [3, 5, 6, 9, 8, 10, 15, 12, 16] After Partition: [3, 5, 6, 9, 8, 10, 15, 12, 16] After Partition: [3, 5, 6, 8, 9, 10, 15, 12, 16] After Partition: [3, 5, 6, 8, 9, 10, 12, 15, 16] Sorted Array: [3, 5, 6, 8, 9, 10, 12, 15, 16] === Code Execution Successful === </pre>

6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the

array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array

accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays

formed. Display the array after each recursive call until the entire array is sorted. Execute your code

and show the sorted array.

CODE

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[len(arr)//2]  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
    result = quick_sort(left) + middle + quick_sort(right)  
    print(result)  
    return result  
  
arr = [19,72,35,46,58,91,22,31]  
print("Original:", arr)  
sorted_arr = quick_sort(arr)  
print("Sorted:", sorted_arr)
```

OUTPUT

main.py	Run	Output
<pre>1- def quick_sort(arr): 2- if len(arr) <= 1: 3- return arr 4- 5- pivot = arr[len(arr)//2] 6- 7- left = [x for x in arr if x < pivot] 8- middle = [x for x in arr if x == pivot] 9- right = [x for x in arr if x > pivot] 10- 11- result = quick_sort(left) + middle + quick_sort(right) 12- print(result) 13- return result 14- 15- arr = [19,72,35,46,58,91,22,31] 16- 17- print("Original:", arr) 18- sorted_arr = quick_sort(arr) 19- print("Sorted:", sorted_arr)</pre>		<pre>Original: [19, 72, 35, 46, 58, 91, 22, 31] [31, 35] [19, 22, 31, 35] [19, 22, 31, 35, 46] [72, 91] [19, 22, 31, 35, 46, 58, 72, 91] Sorted: [19, 22, 31, 35, 46, 58, 72, 91] === Code Execution Successful ===</pre>

7. Implement the Binary Search algorithm in a programming language of your choice and test it on the

array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and

provide the index of the element 20. Modify your implementation to count the number of

comparisons made during the search process. Print this count along with the result.

CODE

```
def binary_search(arr, key):
    low = 0
    high = len(arr)-1
    comparisons = 0
    while low <= high:
        comparisons += 1
        mid = (low + high)//2
        if arr[mid] == key:
            return mid + 1, comparisons
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comparisons

arr = [5,10,15,20,25,30,35,40,45]
key = 20
pos, count = binary_search(arr,key)
print("Position:", pos)
print("Comparisons:", count)
```

OUTPUT

main.py	Output
<pre>1 def binary_search(arr, key): 2 low = 0 3 high = len(arr)-1 4 comparisons = 0 5 6 while low <= high: 7 comparisons += 1 8 mid = (low + high)//2 9 10 if arr[mid] == key: 11 return mid + 1, comparisons 12 elif arr[mid] < key: 13 low = mid + 1 14 else: 15 high = mid - 1 16 17 return -1, comparisons 18 19 arr = [5,10,15,20,25,30,35,40,45] 20 key = 20 21 22 pos, count = binary_search(arr,key) 23 24 print("Position:", pos) 25 print("Comparisons:", count)</pre>	<pre>Position: 4 Comparisons: 4 === Code Execution Success</pre>

8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element

31 using Binary Search. Show the mid-point calculations and the steps involved in finding the

element. Display, what would happen if the array was not sorted, how would this impact the

performance and correctness of the Binary Search algorithm?

CODE

```
def binary_search_steps(arr,key):
```

```
    low = 0
```

```
    high = len(arr)-1
```

```
    while low <= high:
```

```
        mid = (low+high)//2
```

```
        print("Low =",low,
```

```
              "High =",high,
```

```
              "Mid =",mid,
```

```

        "Value =",arr[mid])
    if arr[mid]==key:
        print("Found at Position:",mid+1)
        return
    elif arr[mid]<key:
        low=mid+1
    else:
        high=mid-1
print("Not Found")
arr=[3,9,14,19,25,31,42,47,53]
binary_search_steps(arr,31)
print("\nBinary Search requires sorted array.")

```

OUTPUT

main.py	Output
<pre> 1 def binary_search_steps(arr,key): 2 low = 0 3 high = len(arr)-1 4 5 while low <= high: 6 mid = (low+high)//2 7 print("Low =",low, 8 "High =",high, 9 "Mid =",mid, 10 "Value =",arr[mid]) 11 12 if arr[mid]==key: 13 print("Found at Position:",mid+1) 14 return 15 16 elif arr[mid]<key: 17 low=mid+1 18 else: 19 high=mid-1 20 21 print("Not Found") 22 23 arr=[3,9,14,19,25,31,42,47,53] 24 25 binary_search_steps(arr,31) 26 </pre>	<pre> Low = 0 High = 8 Mid = 4 Value = 25 Low = 5 High = 8 Mid = 6 Value = 42 Low = 5 High = 5 Mid = 5 Value = 31 Found at Position: 6 Binary Search requires sorted array. === Code Execution Successful === </pre>

9. Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and

an integer k , return the k closest points to the origin $(0, 0)$.

CODE

```
import heapq

def kClosest(points,k):
    return heapq.nsmallest(
        k,
        points,
        key=lambda p:p[0]**2+p[1]**2
    )

points=[[1,3],[-2,2],[5,8],[0,1]]
k=2
print(kClosest(points,k))
```

OUTPUT

main.py	Output
<pre>1 import heapq 2 3 def kClosest(points,k): 4 return heapq.nsmallest(5 k, 6 points, 7 key=lambda p:p[0]**2+p[1]**2 8) 9 10 points=[[1,3],[-2,2],[5,8],[0,1]] 11 k=2 12 13 print(kClosest(points,k))</pre>	<pre>[[0, 1], [-2, 2]] === Code Execution</pre>

10. Given four lists A, B, C, D of integer values, Write a program to compute how many tuples

$n(i, j, k, l)$ there are such that $A[i] + B[j] + C[k] + D[l]$ is zero.

CODE

```
from collections import defaultdict
```

```
def fourSumCount(A,B,C,D):
```

```

m=defaultdict(int)
for a in A:
    for b in B:
        m[a+b]+=1
count=0
for c in C:
    for d in D:
        count+=m[-(c+d)]
return count
A=[1,2]
B=[-2,-1]
C=[-1,2]
D=[0,2]
print(fourSumCount(A,B,C,D))

```

OUTPUT

main.py	Output
<pre> 1 from collections import defaultdict 2 3 def fourSumCount(A,B,C,D): 4 m=defaultdict(int) 5 6 for a in A: 7 for b in B: 8 m[a+b]+=1 9 10 count=0 11 12 for c in C: 13 for d in D: 14 count+=m[-(c+d)] 15 16 return count 17 18 A=[1,2] 19 B=[-2,-1] 20 C=[-1,2] 21 D=[0,2] 22 23 print(fourSumCount(A,B,C,D)) </pre>	<pre> 2 === Code Execut </pre>

11. To Implement the Median of Medians algorithm ensures that you handle the worst-case

time complexity efficiently while finding the k-th smallest element in an unsorted array.

CODE

```
def median_of_medians(arr,k):
    if len(arr)<=5:
        return sorted(arr)[k-1]
    groups=[arr[i:i+5] for i in range(0,len(arr),5)]
    medians=[sorted(g)[len(g)//2] for g in groups]
    pivot=median_of_medians(medians,(len(medians)+1)//2)
    lows=[x for x in arr if x<pivot]
    highs=[x for x in arr if x>pivot]
    pivots=[x for x in arr if x==pivot]
    if k<=len(lows):
        return median_of_medians(lows,k)
    elif k<=len(lows)+len(pivots):
        return pivot
    else:
        return median_of_medians(
            highs,
            k-len(lows)-len(pivots)
        )

arr=[12,3,5,7,19]
k=2
print(median_of_medians(arr,k))
```

OUTPUT

```
main.py  [ ] [ ] [ ] Share Run Output
1 def median_of_medians(arr,k):
2
3     if len(arr)<=5:
4         return sorted(arr)[k-1]
5
6     groups=[arr[i:i+5] for i in range(0,len(arr),5)]
7
8     medians=[sorted(g)[len(g)//2] for g in groups]
9
10    pivot=median_of_medians(medians,(len(medians)+1)//2)
11
12    lows=[x for x in arr if x<pivot]
13    highs=[x for x in arr if x>pivot]
14    pivots=[x for x in arr if x==pivot]
15
16    if k<=len(lows):
17        return median_of_medians(lows,k)
18
19    elif k<=len(lows)+len(pivots):
20        return pivot
21
22    else:
23        return median_of_medians(
24            highs,
25            k-len(lows)-len(pivots)
26        )
```

12. To Implement a function median_of_medians(arr, k) that takes an unsorted

CODE

```
def median_of_medians(arr,k):
    if len(arr)<=5:
        return sorted(arr)[k-1]
    groups=[arr[i:i+5] for i in range(0,len(arr),5)]
    medians=[sorted(g)[len(g)//2] for g in groups]
    pivot=median_of_medians(medians,(len(medians)+1)//2)
    left=[x for x in arr if x<pivot]
    equal=[x for x in arr if x==pivot]
    right=[x for x in arr if x>pivot]

    if k<=len(left):
        return median_of_medians(left,k)
```

```

elif k<=len(left)+len(equal):
    return pivot

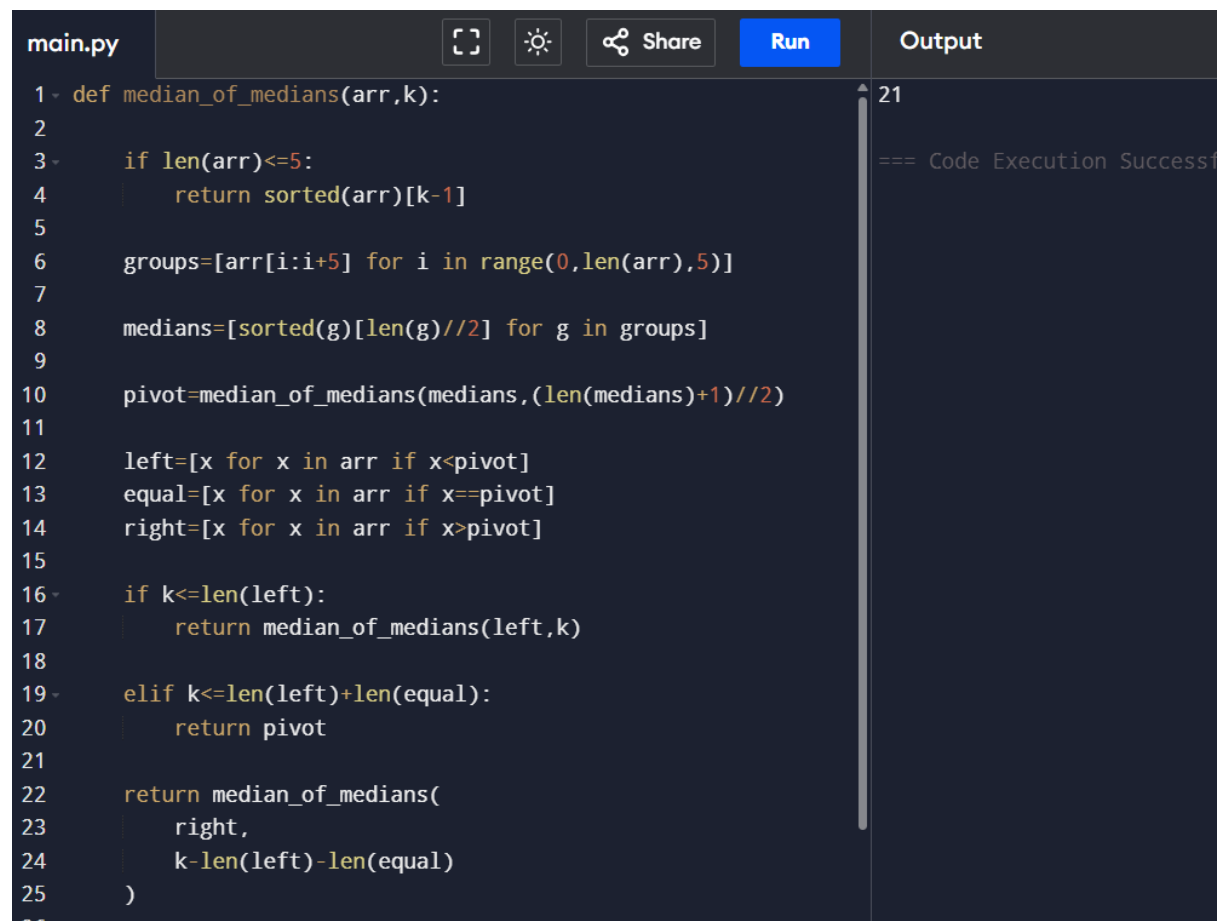
return median_of_medians(
    right,
    k-len(left)-len(equal)
)

arr=[23,17,31,44,55,21,20,18,19,27]

print(median_of_medians(arr,5))

```

OUTPUT



```

main.py
1 def median_of_medians(arr,k):
2
3     if len(arr)<=5:
4         return sorted(arr)[k-1]
5
6     groups=[arr[i:i+5] for i in range(0,len(arr),5)]
7
8     medians=[sorted(g)[len(g)//2] for g in groups]
9
10    pivot=median_of_medians(medians,(len(medians)+1)//2)
11
12    left=[x for x in arr if x<pivot]
13    equal=[x for x in arr if x==pivot]
14    right=[x for x in arr if x>pivot]
15
16    if k<=len(left):
17        return median_of_medians(left,k)
18
19    elif k<=len(left)+len(equal):
20        return pivot
21
22    return median_of_medians(
23        right,
24        k-len(left)-len(equal)
25    )
26
21
=== Code Execution Successful

```

13. Write a program to implement Meet in the Middle Technique. Given an array of integers

and a target sum, find the subset whose sum is closest to the target. You will use the Meet

in the Middle technique to efficiently find this subset.

CODE

```

from itertools import combinations

def subset_sums(nums):
    sums=[]
    n=len(nums)
    for r in range(n+1):
        for comb in combinations(nums,r):
            sums.append(sum(comb))
    return sums

def closest_subset(arr,target):
    mid=len(arr)//2

    left=subset_sums(arr[:mid])
    right=subset_sums(arr[mid:])

    best=None
    for l in left:
        for r in right:
            total=l+r

            if best is None or abs(target-total)<abs(target-best):
                best=total
    return best

arr=[45,34,4,12,5,2]

print(closest_subset(arr,42))

```

OUTPUT

```
main.py  [ ] [ ] [ ] Share Run Output
1 from itertools import combinations
2
3 def subset_sums(nums):
4     sums=[]
5     n=len(nums)
6
7     for r in range(n+1):
8         for comb in combinations(nums,r):
9             sums.append(sum(comb))
10
11     return sums
12
13 def closest_subset(arr,target):
14
15     mid=len(arr)//2
16
17     left=subset_sums(arr[:mid])
18     right=subset_sums(arr[mid:])
19
20     best=None
21
22     for l in left:
23         for r in right:
24             total=l+r
25
26         if best is None or abs(target-total)<abs(target
```

14. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize

the Meet in the Middle technique to handle the potentially large size of the array. Return

true if there is a subset that sums exactly to E, otherwise return false.

CODE

```
from itertools import combinations

def subset_sums(nums):

    sums=[]

    n=len(nums)

    for r in range(n+1):

        for comb in combinations(nums,r):

            sums.append(sum(comb))

    return sums

def exact_subset(arr,target):
```

```

mid=len(arr)//2
left=subset_sums(arr[:mid])
right=set(subset_sums(arr[mid:]))

```

```

for x in left:
    if target-x in right:
        return True

```

```

return False

```

```

arr=[1,3,9,2,7,12]
print(exact_subset(arr,15))

```

OUTPUT

The screenshot shows a code editor with a dark theme. The editor has a tab labeled 'main.py' and a toolbar with icons for full screen, settings, share, and a blue 'Run' button. The code is as follows:

```

1 from itertools import combinations
2
3 def subset_sums(nums):
4     sums=[]
5
6     n=len(nums)
7
8     for r in range(n+1):
9         for comb in combinations(nums,r):
10             sums.append(sum(comb))
11
12     return sums
13
14 def exact_subset(arr,target):
15
16     mid=len(arr)//2
17
18     left=subset_sums(arr[:mid])
19     right=set(subset_sums(arr[mid:]))
20
21     for x in left:
22         if target-x in right:
23             return True
24
25     return False
26

```

To the right of the code editor is an 'Output' panel. It displays the result 'True' and a message '=== Code Execution Successful'.

15 Given two 2×2 Matrices A and B

A=(1 7 B=(1 3

3 5) 7 5)

Use Strassen's matrix multiplication algorithm to compute the product matrix C such that

C=A×B.

CODE

def strassen(A,B):

 a,b=A[0]

 c,d=A[1]

 e,f=B[0]

 g,h=B[1]

 p1=a*(f-h)

 p2=(a+b)*h

 p3=(c+d)*e

 p4=d*(g-e)

 p5=(a+d)*(e+h)

 p6=(b-d)*(g+h)

 p7=(a-c)*(e+f)

 c11=p5+p4-p2+p6

 c12=p1+p2

 c21=p3+p4

 c22=p1+p5-p3-p7

 return [[c11,c12],[c21,c22]]

A=[[1,7],

 [3,5]]

B=[[1,3],

 [7,5]]

C=strassen(A,B)

for row in C:

```
print(row)
```

OUTPUT

main.py	Run	Output
<pre>1 def strassen(A,B): 2 3 a,b=A[0] 4 c,d=A[1] 5 6 e,f=B[0] 7 g,h=B[1] 8 9 p1=a*(f-h) 10 p2=(a+b)*h 11 p3=(c+d)*e 12 p4=d*(g-e) 13 p5=(a+d)*(e+h) 14 p6=(b-d)*(g+h) 15 p7=(a-c)*(e+f) 16 17 c11=p5+p4-p2+p6 18 c12=p1+p2 19 c21=p3+p4 20 c22=p1+p5-p3-p7 21 22 return [[c11,c12],[c21,c22]] 23 24 A=[[1,7], 25 [3,5]] 26</pre>		<pre>[50, 38] [38, 34] === Code Execution S...</pre>